



Computer Games Development CW208
Technical Design Document
Year IV

[Student Name]
[Student Number]

[Jad Fuhr]
[C00290965]

[12/04/2026]

Contents

Technical Design.....	2
References	2

Technical Design Document

Overview

PaleoPals is a 2D mining and exploration game built in C++ using the SFML 3 multimedia library. The game features:

- A destructible tile-based world with four geological terrain layers
- A player character with mining, movement, and upgrade systems
- A JSON-driven fossil and collectible spawning system
- A Museum Interior with dynamic, data-driven fossil displays
- A Trader menu with upgrade systems
- Three search algorithms working together, a depth first search for excavation, breadth first search for the return trip to the surface and A* for fossil collection

This document describes the software architecture, class design, algorithms, and technical decisions that underpin the system, with a focus on the JSON data pipeline and the upgrade/progression system.

Software Architecture

Class Specifications

Game

- Owns the main SFML window
- Runs the game loop (processEvents -- update --render)
- Manages game states: MainMenu, Gameplay, Paused, Exit
- Holds global systems: Map, Player, TraderMenu, MuseumInterior
- Owns the Paleontologist NPC and creates its behaviour tree at startup
- Forwards newly collected items to the Museum interior each frame

Key Methods:

- `run ()`: fixed-timestep loop
- `processEvents()` input handling
- `update ()`: updates world
- `player, AI, render ()`: draws world and UI
- `setupMap()`: loads map config, generates grid and builds npc behaviour tree

Map

- Loads terrain layers, buildings, and fossil config from JSON at startup
- Generates a tile grid with hardness, HP, and textures derived from JSON data
- Handles tile destruction and crack overlay progression
- Spawns collectibles via FossilManager on tile break
- Draws tiles, buildings, and collectibles
- Provides utility functions for AI and player (`getTileHardness`, `tileToWorld`, `worldToTile`)

Key Data Structures - Tile

- `sprite`
- `crackedSprite`
- `currentHP`
- `layerHardness`
- `crackedFrameIndex`

Key Data Structures -LayerType

- `name`,
- `texture`,
- `hardness` (all loaded from JSON)

Key Data Structures -Map Grid

- `std::vector<Tile> m_tiles`

FossilManager

- Loads all collectible types and dinosaur species data from JSON at startup
- Spawns fossil fragments, amber, and trash collectibles when tiles are destroyed
- Assigns random dinosaur species and piece indices to fossil collectibles
- Tracks which pieces have been collected for each species
- Draws collectibles using view-frustum culling
- Provides lookup for nearby collectibles (used by player pickup radius)

Key Algorithms:

- random drop chance via `std::uniform_int_distribution`
- random dinosaur and piece selection ,
- view-frustum culling for collectibles

Player

- Movement, gravity, jumping with jump force scaled by the jump upgrade
- Mining tiles: via a raycast from the player towards the cursor, damages every tile the ray passes through
- Picking up collectibles within an upgradeable pickup radius
- Upgrades: ray length, ray damage, pickup radius, jump height
- Holds money balance: money comes from both player and npc and is spent by upgrade purchases
- Exposes the `addMoney` and `addCollectedItem` functions so npc deposits money the same way player does

Key Systems:

- Physics integration
- tile collision detection
- Mining animation and crack progression
- inventory and money system

Paleontologist (NPC)

- Operates autonomously, with all decision-making delegated to a behaviour tree ticked once per frame from updateNPC()
- Wanders the surface until it decides to begin an excavation
- Excavates using a randomized DFS, breaking tiles along the generated path
- Navigates back to its starting point using BFS across already-broken tiles
- Retrieves dropped collectibles using A*, targeting the nearest reachable one
- Deposits everything it finds through the Player, so npc pickups add money and registers fossils in the museum on the same code path as player pickups
- Excavation depth is upgradeable through the Trader

Key Structures:

Paths:

- m_miningPath (DFS result),
- m_returnPath (BFS result),
- m_fossilPath (A* result),
- each with its own progress index

Key Structures:

State:

- m_miningStartTile (entry point remembered for the return trip),
- m_returningToSurface,
- m_miningDepthLevel,
- m_npcMiningDamage,
- npcMiningTickDelay

Behaviour Tree Framework

- BTNode is an abstract base declaring a single method, tick(float dt), which returns a BTStatus of Success, Failure, or Running.
- Every node in the tree, composite or leaf, implements this one interface, which is what allows the tree to be assembled from arbitrary combinations at runtime.

Composite Nodes:

- SequenceNode: ticks children in order and requires all of them to succeed. A child returning Failure aborts the whole sequence and resets it to the first child. A child returning Running holds the sequence at that index, so long-running behaviours resume where they left off on the next frame rather than restarting.
- SelectorNode: ticks children in order until one succeeds. A child returning Failure moves on to the next. Used here as the tree root so the NPC always has a fallback if its main routine aborts.

Leaf Nodes:

- BTWanderSurfaceNode: patrols the surface and rolls once per second for a 30% chance to commit to a dig, returning Success when it does
- BTMiningNode: generates and follows the excavation path
- BTReturnToSurfaceNode: follows the return path back to the start point
- BTCollectFossilNode: selects, paths to, and collects dropped collectibles

TraderMenu

- Tabbed interface separating hiring from upgrades
- UI for five player upgrades with dynamic cost display: four player stats and one NPC stat
- Calculates and validates upgrade affordability against player money
- Returns purchase actions to Game as a HireAction enum
- Current build has the Hire Palaeontologist button wired to an empty branch in Game::processEvents()

UI Layout:

- semi-transparent overlay
- centered background pane
- tab buttons
- upgrade buttons with live cost and current level text

MuseumInterior

- Displays dinosaur fossil collections assembled from player-collected pieces
- Loads textures for each dinosaur species, skeleton pieces, and cosmetic skins from JSON-defined paths
- Handles navigation between species (arrow buttons)
- Toggles between skeleton and skin display
- Renders interior background, species name, piece progress indicators, arrows, and back button

Key Data Structure

DinoDisplay:

- name
- backgroundTex
- BackgroundSprite.skinTex
- SkinSprite
- pieceTex[4]
- pieceSprite[4]
- collected [4]
- showSkin

JSON Data Pipeline

Overview

All game content is defined in external JSON files and loaded at startup using the `nlohmann::json` library. No content (terrain types, collectible definitions, dinosaur data, building positions) is hardcoded in C++ sources. This design

allows the game to be extended with new species, collectible types, or terrain configurations without recompilation.

JSON File Structure

config.json

- layers []: array of terrain layer objects: name, texture path, hardness value
- museum {}: texture path, frame dimensions, world position
- trader {}: texture path, frame dimensions, world position
- collectibles []: array of all collectible type definitions
- dinosaurs []: array of all dinosaur species definitions

Collectibles Schema

Each entry in collectibles [] defines index, name, type (fossil/amber/trash), texture path, frameWidth , frameHeight , frameIndex , monetaryValue

Dinosaurs Schema

Each entry in dinosaurs [] defines name, category (Carnivore/Herbivore), background texture path, skin texture path, pieces [] - each piece has: id (string) and texture path

Loading and Parsing

Map::setupMap() opens map.json, parses it with nlohmann::json, and populates the LayerType list, building configs, and passes the collectibles and dinosaurs arrays to FossilManager.

FossilManager iterates the collectibles [] and dinosaurs [] arrays using range-based for loops, constructing CollectibleDef and DinosaurDef structs at runtime. Texture paths are loaded into sf::Texture objects. Piece assignments are stored as vectors indexed by dinosaur index.

MuseumInterior receives the parsed dinosaur data and loads each species' background, skin, and piece textures into DinoDisplay structs for rendering.

Why nlohmann::json

- Header-only: no build system changes required
- STL-compatible API: iterates like std::vector, accesses like std::map
- Supports complex nested structures (arrays within objects within arrays)
- Type-safe value access with .get<T>() and operator[]
- Easy to modify content files without touching C++ code

AI System

The palaeontologist NPC is fully autonomous. Rather than a state machine with hand-written transitions, its behaviour is expressed as a behaviour tree built once at startup in `Game::setupMap()` and ticked every frame. Each frame the tree is walked from the root, and the first node that returns `Running` takes ownership of that frame.

Tree Structure:



The routine is a sequence because every step depends on the one before it: there is nothing to excavate before committing to a dig, nothing to collect before excavating, and no way home before something has been dug out.

The sequence returning `Failure` at any point resets the NPC to the top of the routine, which is the recovery mechanism described below.

`BTReturnToSurfaceNode` appears twice deliberately. The first instance carries the NPC out of the tunnel it just dug; the second runs after collection, once the NPC has wandered off the return path chasing collectibles.

Path Generation

Three different searches are used because the three navigation problems have genuinely different constraints.

Excavation: randomised DFS

The NPC is not navigating existing terrain, it is choosing which terrain to dig up. I believe DFS suits this because it drives hard in one direction rather than spreading evenly, producing tunnels rather than caverns.

Neighbours are shuffled before being pushed so successive digs do not look identical.

The search is capped by a depth budget:

$$\text{maxDepth} = 10 + \text{m_miningDepthLevel} * 10$$

This is the value the Trader upgrade increases by, and it is what ties the npc directly into the economy: money spent on the npc produces longer tunnels and therefore more drops.

Return trip: BFS

The NPC must get home along tiles that have already been mined, and it should not take a long scenic route.

BFS on an unweighted grid guarantees the shortest such path, and the terrain graph is small enough that its cost is irrelevant.

A parent grid records how each tile was reached, and the path is reconstructed backwards from the goal and reversed.

Fossil retrieval: A*

- Collectibles are scattered across the excavated area rather than sitting at a single known destination, so the search runs repeatedly with a different goal each time. A* with a Manhattan heuristic is admissible on a 4 connected grid and directs the search toward the target instead of expanding blindly in all directions, which matters when the same search runs once per collectible.
- Open nodes are held in a `std::priority_queue` ordered by the f-score.
- Visited nodes are stored in a `std::map` keyed by tile position, which requires a custom comparator function since `sf::Vector2i` does not have any ordering of its own.

Failure Handling

Autonomous agents fail in ways scripted ones don't, so each leaf node needs some form of failure handling:

- Excavation deadlock. If the NPC finishes excavating but no return path can be generated, BTMiningNode clears its state and returns Failure, resetting the tree instead of leaving the NPC stuck.
- Trapped underground. BTReturnToSurfaceNode runs a ten-second timer. If it expires with no path, the node fully resets the NPC's state and fails the sequence so the routine restarts.
- Unreachable collectibles. A collectible that A* cannot reach is not abandoned permanently. BTCollectFossilNode has a collectible attempt counter and only skips one after three failures. The counters are cleared on every successful pickup, because collecting proves the NPC has moved and the dig site has opened up, making earlier failures redundant. Before the node reports that it is finished, it clears the counters once and goes again, so a collectible is only left behind when it has failed a full second round.
- Stalled pursuit. A five-second timeout on the current target counts as a failed attempt and moves the NPC on, covering the scenario where a path exists but the NPC cannot physically complete it.

Collection Pipeline

- When the NPC reaches a collectible it builds a CollectedItem and calls Player::addMoney() and Player::addCollectedItem().
- This pushes the item onto the same m_newPickups list the player uses, which Game::update() forwards to the Museum Interior.
- Routing NPC pickups through the Player rather than giving the NPC its own inventory means the money calculation, the museum notification, and the fossil/amber/trash classification all exist in one place.

Upgrade System

Overview

The Trader menu exposes four upgradeable player stats. Upgrades are purchased using in-game money earned from selling fossils and amber. Each upgrade applies immediately to the relevant player system.

Upgrade Definitions

- Length: increases ray length (tiles affected by intersecting with ray)
- Damage: increases damage dealt if intersecting with ray
- Pickup: increases the radius within which collectibles are auto collected
- Jump: increases maximum jump height (affects gravity/velocity parameters)
- NPC Mining Depth: increases how far the npc will mine

Cost Calculation

- Length: $100 + \text{level} \times 50$
- Damage: $200 + \text{level} \times 75$
- Pickup: $150 + \text{level} \times 60$
- Jump: $175 + \text{level} \times 70$
- NPC Mining Depth: $500 + \text{level} \times 100$

Costs are checked against the player's balance in `Game::processEvents()` before the stat is incremented, so an unaffordable click does not work

The NPC depth upgrade is deliberately the most expensive. It is the only upgrade that increases income passively rather than improving the player's own reach, so it sits as a purchase once the player has spent a lot of time mining themselves

Economy Integration

Every collectible type carries a monetaryValue defined in JSON, so fossils, amber and trash earn you money, with fossils additionally contributing a piece to a museum display.

Collection by the NPC adds money to player the same way it does when player is the one that does it, which turns the depth upgrade into a passive income stream. Money is spent at the Trader, and the balance is always displayed in the HUD.

Algorithms

Tile Mining

damageTile(row, col, dmg): t.currentHP -= dmg - update crackedFrameIndex -
if currentHP <= 0: removeTile(row, col) - FossilManager.trySpawn(row, col)

Fossil Spawn

trySpawn(row, col): if random (0–99) < spawnChance - choose collectible type
from weighted list, if fossil: assign random dinosaur index and piece index -
push to m_collectibles

Upgrade Application

onUpgradePurchase(type): deduct cost from player.money - increment player
stat, recalculate next upgrade cost - refresh Trader UI button text

DFS Excavation

- generateMiningPath():
- push start tile onto stack
- while stack not empty and path size < (20 + depthLevel * 10):
- pop tile from stack
- skip if out of bounds or already visited
- mark visited, append to mining path
- shuffle the four neighbours
- push unvisited neighbours

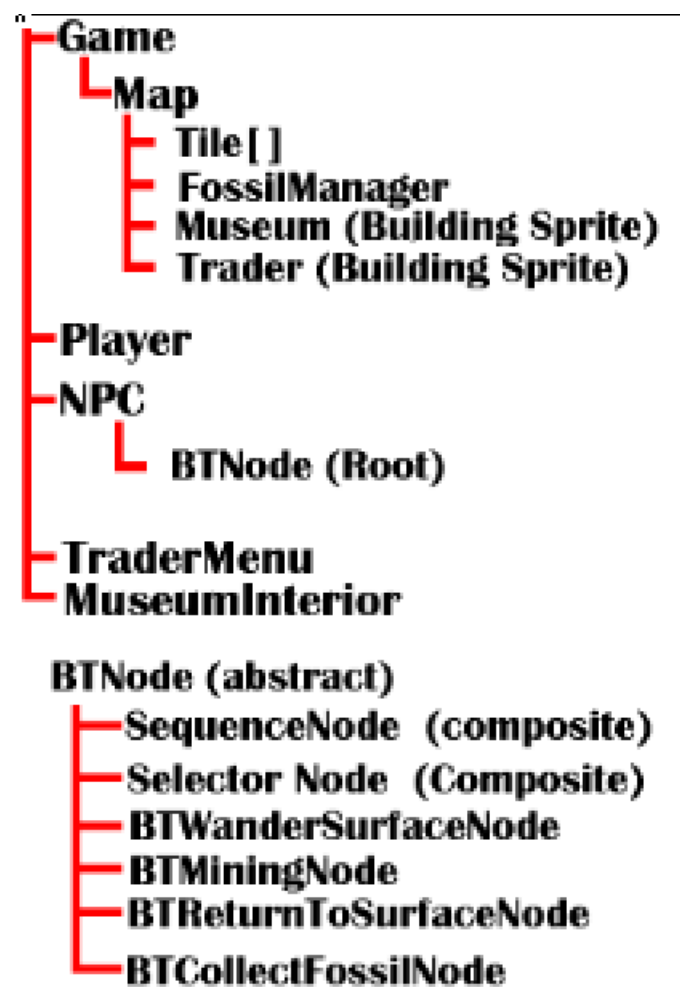
BFS Return Path

- generateReturnPath():
- push current tile into queue, mark visited
- while queue not empty:
- pop tile
- stop if it is the entry point
- for each neighbour that is broken and unvisited:
- record parent,
- mark visited,
- enqueue
- walk parents backwards from the entry point and reverse

A* Fossil Path

- generateFossilPath(goal):
- push start with $f = \text{manhattan}(\text{start}, \text{goal})$
- while open not empty:
- pop lowest f
- stop if it is the goal
- for each broken neighbour:
- $g = \text{current } g + 1$
- $h = \text{manhattan}(\text{neighbour}, \text{goal})$
- if unvisited or g improves on the recorded cost: record parent and costs, push with $f = g + h$
- walk parents backwards from the goal and reverse

UML Class Diagram (Simplified)



Design Decisions

Why SFML

- Lightweight and easy to set up
- Simple, clean rendering pipeline for 2D
- Good sprite, texture, and view management for tile-based games

Why JSON (nlohmann::json)

- All content (terrain, fossils, dinosaurs) modifiable without recompilation
- Nested structure support handles complex dinosaur piece data cleanly
- nlohmann::json integrates naturally with C++ STL types
- Header-only - no additional build dependencies

Why Data-Driven over Hardcoded

- Faster iteration: change a hardness value or add a dinosaur without touching code
- Cleaner separation of content and logic
- Easier to extend new species, collectible types, or terrain layers require only JSON edits

Why a Behaviour Tree over a State Machine

- Nodes are self-contained and testable in by themselves without knowing what comes before or after them
- The routine can be reordered or expanded upon by changing how they are assembled in setupMap() rather than editing transition logic across states
- Running tasks are inherently modeled in behavior trees, since multi-frame actions can simply remain active until completion. In contrast, a state machine requires extra bookkeeping to track whether something is still in progress. Failure handling flows upwards automatically in a behaviour tree, so recovery is determined by the structure of the tree itself rather than something every state must handle individually

Why Three Search Algorithms

Each navigation problem has different issues or constraints that may arise, and using one general algorithm everywhere would either waste work or produce the wrong behaviour:

- Excavation wants a tunnel, not an optimal route. DFS gives that shape for free, a shortest-path search would produce a straight line with no character
- The return trip wants guaranteed shortest on an unweighted graph, which is exactly BFS with no heuristic to justify
- Fossil retrieval runs repeatedly against scattered goals, so the heuristic guidance A* provides pays for its extra complexity

Implementing all three also demonstrates the trade offs between them rather than applying one algorithm uniformly.

Future Technical Improvements

- Formal JSON schema validation at load time to catch malformed config errors
- Multi-tier upgrade tree with branching paths
- Save/load system that persists with player money, upgrades, and museum progress
- Procedural fossil distribution using JSON-defined rarity weights
- Multiple simultaneous palaeontologists with a shared claim system, so two NPCs do not path to the same collectible
- Specialised NPC roles built by composing different behaviour trees from the existing node set
- Path caching between collection targets, since consecutive A* searches from nearby positions repeat most of their work

References

Videos Used

Ögren, P. (2021). Behavior trees playlist [YouTube playlist]. YouTube.
https://www.youtube.com/playlist?list=PLFQdM4LOGDr_vYJuo8YTRcmv3FrwczdKg

Computerphile. (2014, December 18). A search algorithm* [Video]. YouTube.
<https://www.youtube.com/watch?v=ySN5Wnu88nE>

Shah, M. (2020). C++ programming playlist [YouTube playlist]. YouTube.
<https://www.youtube.com/playlist?list=PLvv0ScY6vfd8j-tlhYVPYgiIyXduu6m-L>

Books Used

Meyers, S. (1996). More effective C++: 35 new ways to improve your programs and designs. Addison-Wesley.

Documentation Used

SFML Team. (2024). SFML 3.0.0 documentation. SFML. <https://www.sfml-dev.org/documentation/3.0.0/>

Some Quick Reads that Helped:

ScholarWorks@GVSU, “Behavior Trees in Robotics and AI,” Accessed: Aug. 30, 2026. [Online]. Available:
<https://scholarworks.gvsu.edu/cgi/viewcontent.cgi?article=1367&context=cistechlib>

Lisyrus, “Behavior Trees,” Accessed: Aug. 30, 2026. [Online]. Available:
<https://lisyrus.github.io/blog/posts/behavior-trees.html>

Websites Used

Nlohmann. (2024). basic_json API documentation. JSON for Modern C++. https://json.nlohmann.me/api/basic_json/

Colledanchise, M. (2026). BehaviorTree.CPP source code. GitHub. <https://github.com/BehaviorTree/BehaviorTree.CPP/tree/master/src>

Colledanchise, M. (2026). Awesome Behavior Trees. GitHub. <https://github.com/BehaviorTree/awesome-behavior-trees>